

UNIT 2

BASICS OF PROGRAMMING IN C++



Q.1: DEFINE COMPUTER PROGRAM?**COMPUTER PROGRAM**

A computer program is a set of instructions that performs a specific task when executed by a computer. A computer requires programs to function and typically executes the program's instructions in a central processing unit. A computer program is usually written by a computer programmer in a programming language.

Q.2: DEFINE SYNTAX OR WHAT IS SYNTAX IN PROGRAMMING?**SYNTAX IN PROGRAMMING LANGUAGE**

Syntax refers to the rules that define the structure of a language. Syntax in computer programming means a set of keywords and characters that a computer can understand, interpret and perform task associated with them. The syntax of a language must be followed, and if it is not followed, the code will not be understood by a compiler or interpreter. Different programming languages have different types of syntax.

Q.3: DESCRIBE CLASSIFICATION OF PROGRAMMING LANGUAGE.**CLASSIFICATION OF PROGRAMMING LANGUAGE**

Based on the accessibility of hardware, programming languages can be classified into the following categories:

1. Low-level language
2. Middle-level language
3. High-level language

Q.4: DEFINE LOW LEVEL LANGUAGE.**LOW-LEVEL LANGUAGE**

Low-level programming languages are those languages that are directly communicated with computer hardware. The two languages come under this category are Machine language and Assembly language.

Q.5: DEFINE MACHINE LANGUAGE.**MACHINE-LEVEL LANGUAGE**

Machine language is a collection of binary digits or bits that the computer reads and interprets. This language is the only language understood by computers while machine language is almost impossible for humans to use because they consist entirely of numbers (0s & 1s). Machine code doesn't require translator because machine code is directly executed by computer. It is also called first generation language.

Q.6: DEFINE ASSEMBLY LANGUAGE.**ASSEMBLY LANGUAGE**

A program written in assembly language consists of a series of instructions called mnemonics that correspond to a stream of executable instructions. Assembly language code is translated by a translator called assembler. Assembly language uses keywords and symbols much like English and are easy to read, write and maintain as compared to machine language. It is also called second generation programming language.

Q.7: DEFINE MIDDLE LEVEL LANGUAGE.**MIDDLE-LEVEL LANGUAGE**

The middle-level language lies in between the low level and high-level language. Middle-level language actually binds the gap between a machine level language and high-

level languages, these languages are now become obsolete and are not in used.

Q.8: WHAT DO YOU KNOW ABOUT HIGH LEVEL LANGUAGE?

HIGH-LEVEL LANGUAGE

High-level languages are relatively new and drastically revolutionized the programming world. It allows us to write computer code using instructions resembling everyday spoken language, usually English (for example: print, if, while). Programs written in a high-level language need to be translated by a translator (compiler or interpreter) into machine language before execution.

Q.9: WRITE DOWN THE ADVANTAGES OR BENEFITS OF HIGH-LEVEL LANGUAGE.

ADVANTAGES OF HIGH-LEVEL LANGUAGE

- High level language is much closer to human language so it is more suitable to write code in high level language.
- It is more or less independent or portable of the particular type of computer used.
- It is easier to read, write and maintain.

Q.10: WRITE DOWN THE DIFFERENCES BETWEEN LOW-LEVEL AND HIGH-LEVEL LANGUAGES.

DIFFERENCES BETWEEN LOW-LEVEL LANGUAGE AND HIGH-LEVEL LANGUAGE

Low-level languages	High-level languages
It is a machine-friendly language, i.e., the computer understands the machine language, which is represented in 0 or 1.	It is a user-friendly language as this language is written in simple English words, which can be easily understood by humans.
It requires the assembler to convert the assembly code into machine code.	It requires the compiler or interpreter to convert the high-level language instructions into machine code.
One type of machine code cannot run on all machines, so it is not a portable language.	The high-level code can be translated to required machine code, so it is a portable language.
It has direct access to memory.	It is less memory efficient.
Coding and maintenance are not easy in a low-level language.	Coding and maintenance are easier in a high-level language.

Q.11: DEFINE TRANSLATORS OR LANGUAGE TRANSLATORS.

TRANSLATORS OR LANGUAGE TRANSLATORS

The examples of high-level languages are: BASIC, FORTRAN, COBOL, PASCAL, C languages etc. Language translator is a computer program that converts high level language program into low level program (1s & 0s), or machine language. It transforms the source code into the object code (machine code) which understands directly by the computer processor. Translators also detect and report errors in the process of translation. There are three types of language translators.

- Interpreter
- Compiler
- Assembler



Q.12: WHAT IS COMPILER?

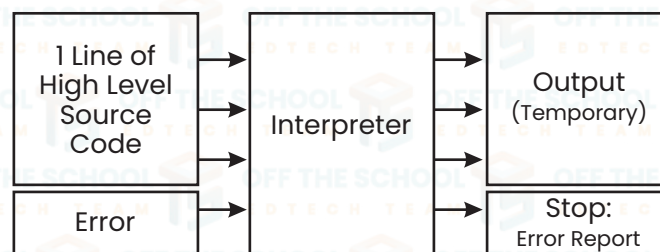
COMPILER

Compiler is a program that translates the high-level language program into machine language. Compiler translates the whole program at a time at once before it executed and makes a separate object file for the translated program. Translated program can be used multiple times without the need of retranslation of source codes. Each high-level language has its own compiler.

Q.13: DEFINE INTERPRETER?

INTERPRETER

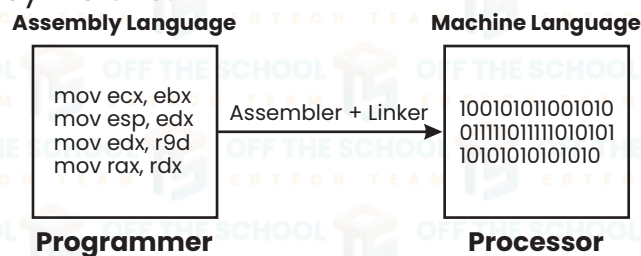
Interpreter is a language translator program, which converts high level program into machine language. It translates one instruction at a time. Interpreter does not make any object file and it translates the program every time when executed. An interpreter is faster than a compiler as it immediately executes the codes.



Q.14: DEFINE ASSEMBLER.

ASSEMBLER

An assembler is a translator that converts assembly language program into machine language. An assembler translates assembly language code directly into machine code that can be understood by the CPU.



Q.15: DEFINE SOURCE PROGRAM OR SOURCE CODE?

SOURCE PROGRAM/SOURCE CODE

A program or A program written by a programmer in any language other than machine language is called a source program or source code.

Q.16: DEFINE OBJECT PROGRAM OR OBJECT CODE.

OBJECT PROGRAM/OBJECT CODE

Object program is a program or code that is converted into machine language. (OR) The output from a language translator, which consists of machine language instructions, is called the object program.

Q.17: WRITE DOWN THE DIFFERENCES BETWEEN INTERPRETER AND COMPILER?

DIFFERENCES BETWEEN INTERPRETER AND COMPILER

Feature	Compiler	Interpreter
Process	Translates entire program at once	Translates code line by line during execution
Output	Machine code (executable file)	No output; interprets source code directly
Speed of Execution	Faster (optimized machine code)	Slower (needs translation on the fly)
Error Detection	Shows all errors after compilation	Shows errors line by line during interpretation
Development Efficiency	May require recompiling for changes	Easier debugging; changes don't require recompiling
Memory Usage	Higher (stores compiled code)	Lower (no need to store machine code)
Examples	C, C++, Java, Scala	Python, JavaScript, Ruby, PHP

Q.18: DEFINE ERRORS OR PROGRAMMING ERRORS.**ERROR**

An error describes any issue that arises unexpectedly that causes a program to not function properly. In general, there are three types of errors that occur in a computer program. Syntax Error, Logical Error and Runtime Error. adamjeeecoaching.blogspot.com

Q.19: WHAT IS SYNTAX ERROR?**SYNTAX ERROR**

A syntax error is an error in the syntax due to violation of rules of whatever language program is being written. These are sometimes caused by simple typing mistakes.

Q.20: WHAT IS LOGIC ERROR?**LOGIC ERROR**

In computer programming, a logic error is a bug or error in planning the program's logic. They do not cause the program to crash or simply not work at all, they cause it to "misbehave" in some way, rendering wrong output of some kind. The computer does not tell you what is wrong.

Q.21: WHAT IS RUN TIME ERROR?**RUNTIME ERROR**

A runtime error is a program error that occurs when a program is run on the computer and the result are not achieved due to some misinterpretation of a particular instruction. The code doesn't have any syntax or logic error but when it executes it cannot perform specific task.

Q.22: WHAT IS PROGRAMMING ENVIRONMENT OF C++?**PROGRAMMING ENVIRONMENT OF C++**

Programming environment is an environment which supports execution of programming language smoothly and efficiently on a local computer to compile and run programs.

Q.23: DEFINE IDE OR INTEGRATED DEVELOPMENT ENVIRONMENT?**INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)**

An integrated development environment (IDE) is software for building applications or programs that combines common developer tools into a single graphical user interface (GUI). IDE facilitates the development of applications designed to encompass all programming tasks in one application, one of the main benefits of an IDE is that they offer a central interface with all the tools, a developer needs. Dev-C++ is used for writing

programs in C++ language, however there are many multiple language IDEs.

Q.24: WRITE DOWN THE BENEFITS OR ADVANTAGES OF INTEGRATED DEVELOPMENT ENVIRONMENT.**BENEFITS/ADVANTAGES OF INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)**

- IDE combines all tools that need for development. Programmers don't need to switch between different tools to design a layout, write the code, debug, build, etc.
- Many IDEs incorporate basic spelling checkers, so automatically check for errors to improve code.
- Libraries provide for functions in IDEs that are not included in the core part of the programming language.

Q.25: WHAT ARE THE COMPONENTS OF INTEGRATED DEVELOPMENT ENVIRONMENT?**COMPONENTS OF INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)**

IDES combining common tools that are necessary for a programmer to develop program,

EDITING SOURCE CODE

Writing and editing source codes is a major part of programming. A text editor is used for writing and editing source codes with feature providing language specific autocompletion, and checking for bugs as code is being written.

SYNTAX HIGHLIGHTING

Syntax highlighting is a feature of IDEs that provides visual cues. keywords, and other words that have special meaning in languages are highlighted. This feature makes code easier to read or understand.

CODE COMPLETION

It is a feature of IDE that completely knows the language syntax that speeds up the process of coding by reducing typos and other mistakes. Autocompletion popups while typing, querying parameters of functions, query hints related to syntax errors.

COMPILER

Compiler is a component of IDEs that translate source code into machine code. IDEs provide automated build process. This feature can help automate developer tasks that are more common to save time.

Q.26: DEFINE LINKER, LOADER AND DEBUGGING?**LINKER**

Linker connects or links referenced library files with comp code. It saved the linked objects into an executable file.

LOADER

This is the operating system's program that loads executable files into memory and directs the CPU to start running the program as directed by the IDE.

DEBUGGING

The process of removing errors from a program is known as debugging.

Q.27: WHAT DO YOU KNOW ABOUT DEV C++?

INTRODUCTION TO DEV-C++

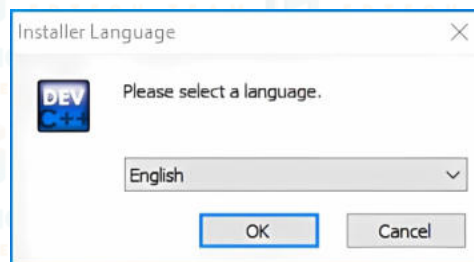
Dev-C++ is a fully featured graphical IDE (Integrated Development Environment) for programming in C/C++. Dev-C++ is developed by Bloodshed software. It was originally developed by Colin Laplace and first released in 1998. It is written in Delphi. With Dev C++ programmer can write Windows or console-based C/C++ programs easily.

Q.28: WRITE DOWN THE INSTALLATION PROCEDURE OF DEV C++.**INSTALLING AND CONFIGURING DEV-C++**

IDE Dev-C++ is free available for download on Internet. After downloading the package begin the installation process. Following are the steps for installing Dev-C++ IDE.

STEP 1

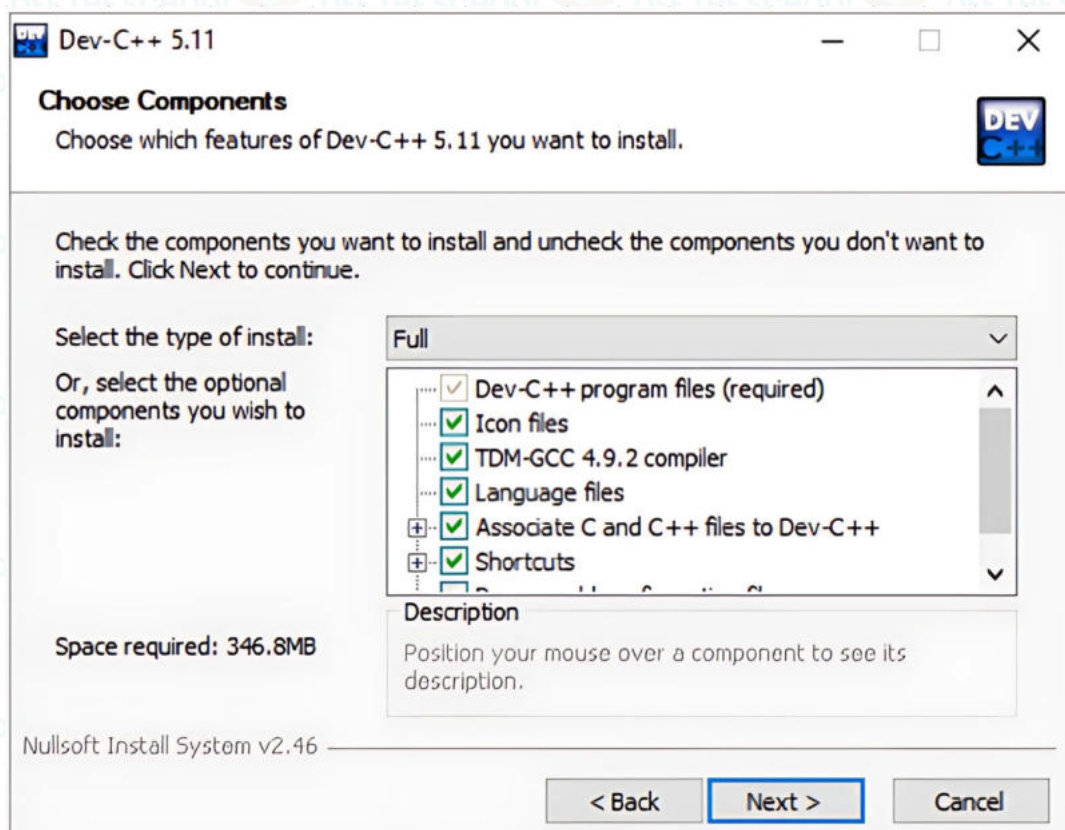
Select "English" as the language to be used for installation process.

**STEP 2**

Agree to the license agreement by pressing "I Agree" button.

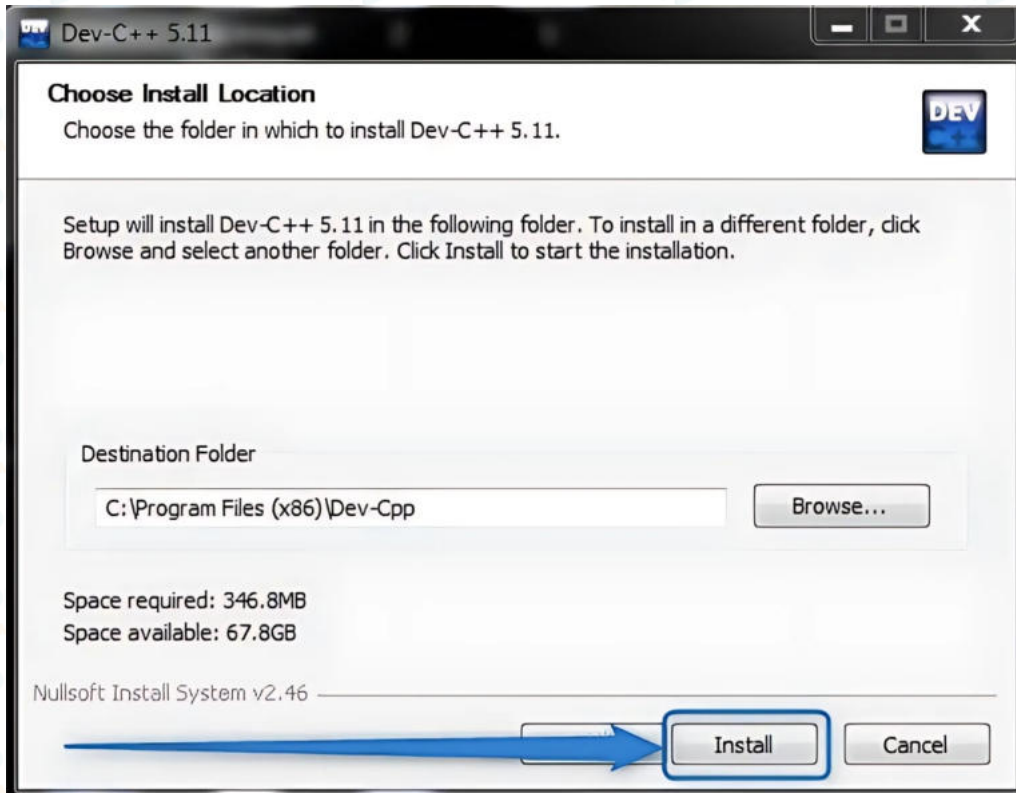
STEP 3

Select "Full" from the dropdown for "type of Install". This will select all the necessary components required to run Dev-C++ and compile C++ source codes. Click on "Next" to proceed.

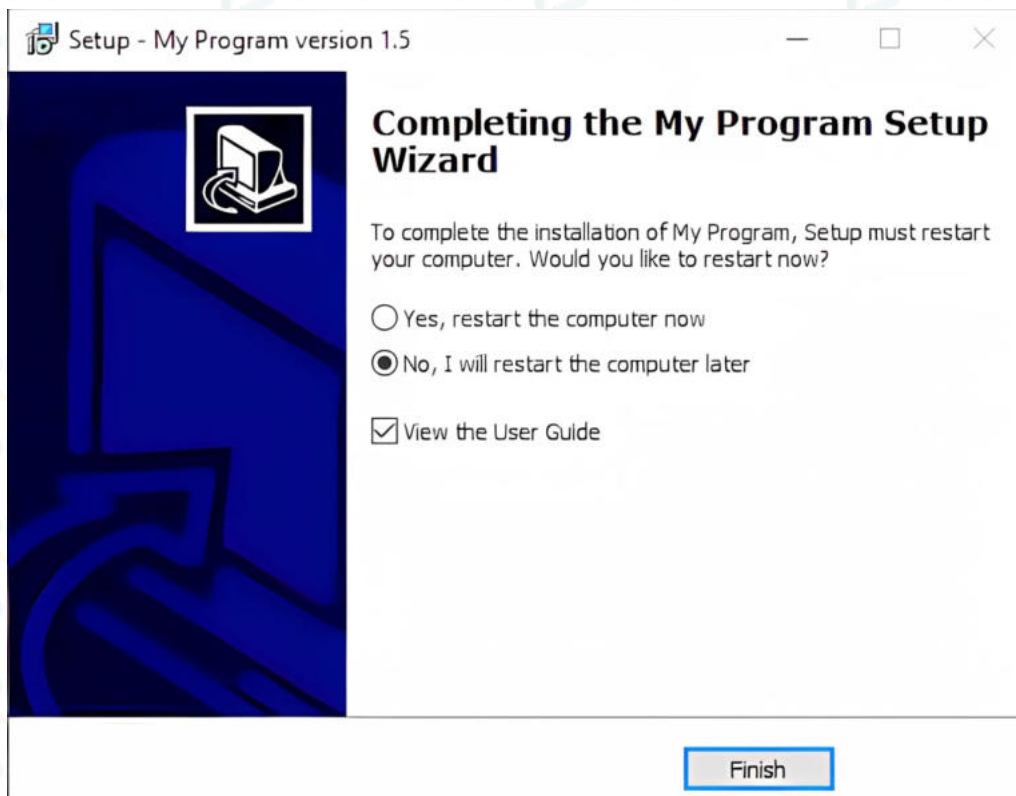


STEP 4

Select the installation directory where all the necessary Dev-C++ files and libraries will be installed. Usually, the default specified path is used for installation but you can change it if desired. Click on “Install” to begin installation.

**STEP 5**

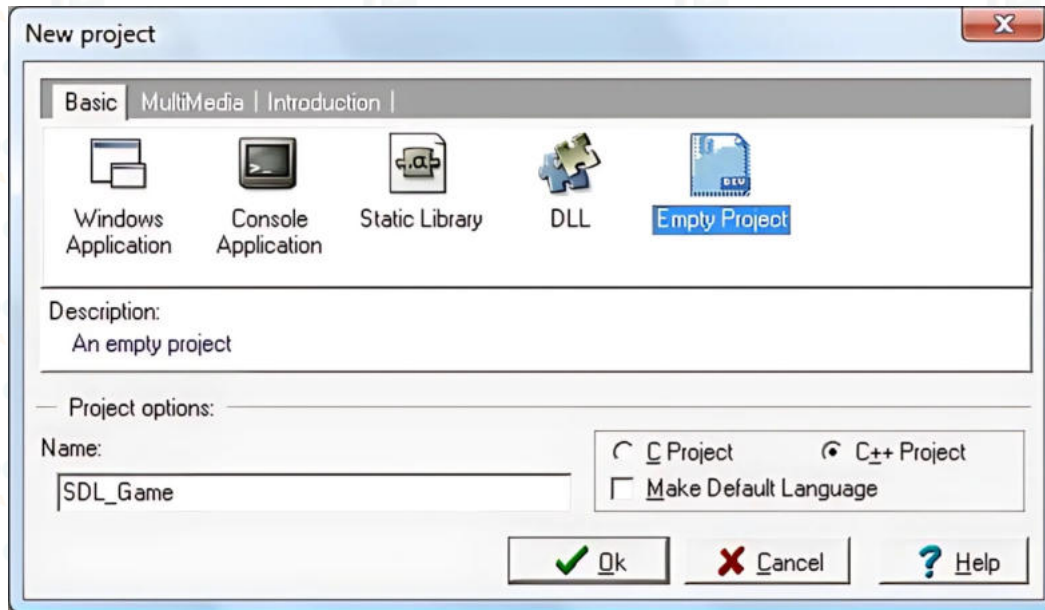
The installer will show the progress for installation. Once the process completes, it will show a “Finish” dialog. Make sure the “Run Dev-C++ 5.11” box is checked. This will automatically start Dev-C++ IDE after this installation completes. Click “Finish” button to complete the installation process.



Q.29: WRITE DOWN THE PROCEDURE TO DEVELOP A PROGRAM IN DEV C++.**DEVELOPING PROGRAM IN DEV C++**

C++ development is done by writing source codes and saving those files for compilation. Dev-C++ provides good project management support to help manage C++ files and group them into projects. The steps to create a new project in Dev-C++ are:

1. Click on **File** → **New** → **Project**.
2. From the **New Project** dialog, make sure **Empty Project** is selected. From language options, select **C++ Project**. Then enter a **Name** for your project.
3. Click on **OK**. Dev-C++ will ask for the path where you want the new project to be stored. Once it is done, Dev-C++ will open a workspace. It will show Project Explorer on the left side that shows the project we just created.

**ADD FILES TO PROJECT**

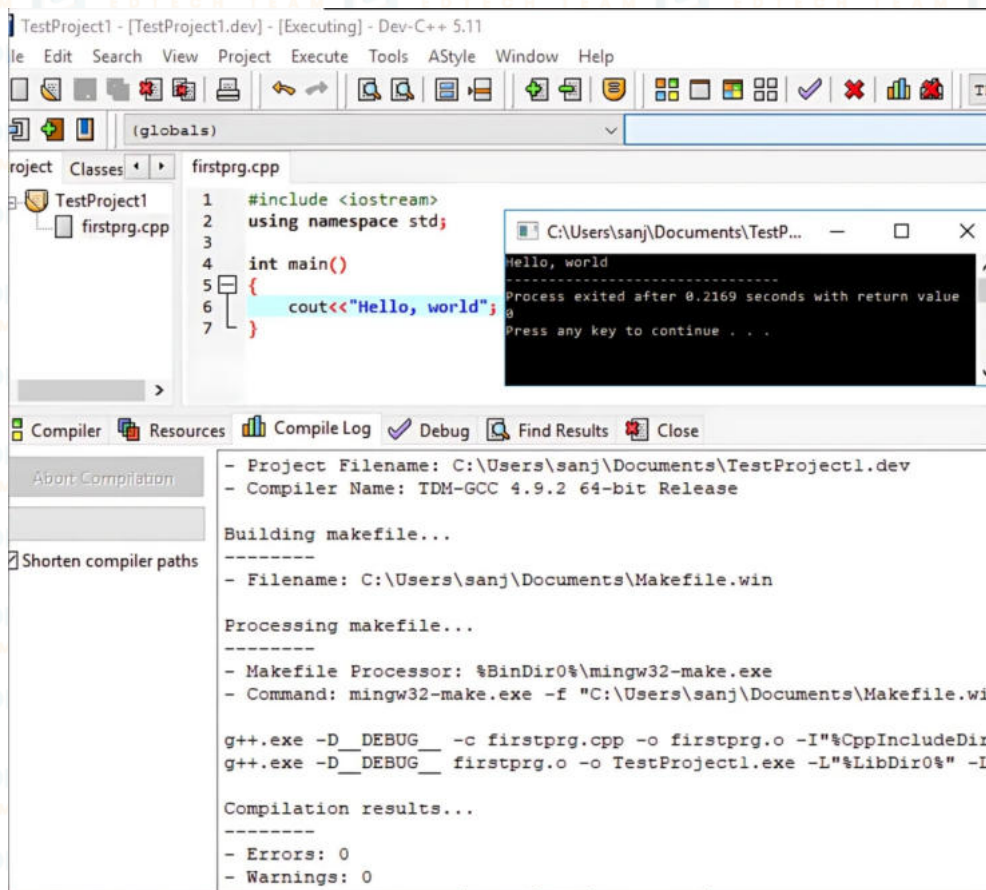
A project requires source files which will contain codes for your program. The steps to create a new file are:

1. Click on Project → New File. Alternatively, you can also right-click on the Project Name in the Project Explorer and click on New File.
2. Click on Yes on the Confirm dialog to add a file. This file is not stored until it is deliberately saved.
3. To save newly added file, click on File → Save. Enter a path where you want to save the file and provide its name. Click on Save to store the file.

COMPILE AND EXECUTE PROJECT

After writing the source codes in files, the project needs to be compiled and executed to see its output. Follow these steps to compile and run a project:

1. The project needs to be compiled before execution. To compile, click on Execute → Compile or press F9 key. Compile Log tab shows the compilation status. Compiler tab will show if there are any syntax errors.
2. After successfully compiling the project, run it by clicking on **Execute** → **Run** or by pressing **F10** key.
3. A console window will open and show the output of the program.



Q.30: DEFINE RESERVED KEYWORDS IN C++.

RESERVED WORDS

A reserved word in C++ is a word whose meaning is already defined by the compiler. A reserved word cannot be used as an identifier, such as the name of a variable, function, or label – it is "reserved from use in C++". A reserved word is part of syntax and may not have any specific meaning in the English language.

There is a total of 95 reserved words in C++. The reserved words of C++ may be conveniently placed into several groups. In the first group, we put those that were also present in the C programming language and have been carried over into C++. There are 32 of these.

There are another 30 reserved words that were not in C, are therefore new to C++ programming language. Some of the commonly used C++ reserved are:

and	break	case
auto	char	do
bool	class	else
catch	const	export
default	continue	float
double	delete	goto
enum	explicit	inline
extern	false	module
for	friend	new
if	import	or
int	long	protected
nullptr	namespace	short
public	not	static
requires	operator	struct

signed	private	template
switch	register	throw
this	return	typedef
true	sizeof	union
unsigned	try	virtual
void	using	while

Q.31: DEFINE SYMBOLIC CONSTANT.**DEFINED OR SYMBOLIC CONSTANT**

In C++, we can create symbolic constant whose value remains unchanged but used as a variable. A symbolic constant can be created using the `#define` preprocessor directive or `const` keyword.

Example:

```
const int LIGHT_SPEED=299792458;
#define LIGHT_SPEED 299792458
```

Q.32: DEFINE VARIABLE AND RULES FOR NAMING VARIABLES.**VARIABLE**

A variable is nothing but a name given to a storage area that our programs can manipulate. Its value can change during program execution. Each variable in C++ has a specific data type, which determines the size and layout of the variable's memory.

RULES FOR NAMING VARIABLE

- A variable name contains alphabets, numbers, and the underscore.
- A variable name must start with a letter or an underscore.
- Variable names are case sensitive. (Sum and sum are different)
- A variable name cannot be a keyword.
- A variable name cannot be longer than 32 characters.

Q.33: DEFINE DECLARING (CREATING) AND INITIALIZING VARIABLES.**DECLARATION (CREATING) VARIABLES**

Variable declaration is a process in which we create storage space for variable in memory. A variable declaration consists of data type and name of the variable written as follow:

Syntax

```
data_type variable_name;
```

INITIALIZATION

Assign initial value to a variable is known as variable initialization. It can be initialized during declaration or separately. The equal sign is used to assign value written as follows:

Syntax

```
data_type variable_name = value;
```

Q.34: DEFINE STRINGS IN C++.**STRINGS IN C++**

Variables that can store alphanumeric value that consist of multiple characters are called strings. In C++, strings are used by one-dimensional array of characters, which is terminated by a null character.